COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	07/16/2026	**DATE SUBMITTED:**	07/16/2026
NAME:	Troy Jerron A. Bautista	**INSTRUCTOR:**	Sir Jimlord Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
    std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
     studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
}
};

int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;
    return 0;
}
```

```
PS C:\Users\TIPQC\.vscode> cd "c:\Users\TIPQC\.vscode\" ; if ($?) { g++ ILO_A.cpp -o ILO_A } ; if ($?) { .\ILO_A }
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation	First, it will start from student1 and put the studentName and studentAge with the details provided (Roman, 28) and output “Constructor Called”. The object of student2 will copy the details of student1, but leave the studentName blank, but copy the studentAge and output “Copy Constructor Called”. The student3 will copy the details that was inside the class Student in public (John Doe, 18) and outputs “Constructor Called”. The next line will copy the contents of student2 to student3 and this will point to the studentName of student2 and the object of student3. Then the object will now start using its destructor, which will delete the content inside the studentName (except for the studentAge) on all of the main and output “Destructor Called”.

Table 2-1. Initial Driver Program

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
    std::cout << "Destructor Called." << std::endl;
}
//Copy Constructor
Student(const Student &copyStudent){
    std::cout << "Copy Constructor Called" << std::endl;
    studentName = copyStudent.studentName;
     studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
}
};

int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};
    return 0;
}
```

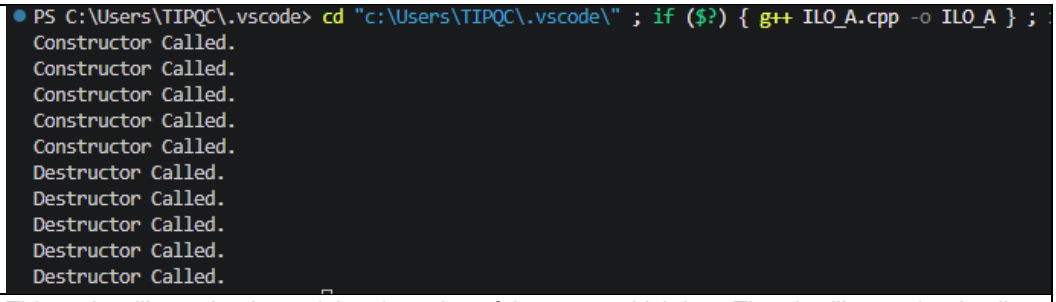
	
Observation	This code will start by determining the value of the array, which is 5. Then it will copy the details that was inside the class Student in public (John Doe, 18) on the studentList and outputs "Constructor Called". The namesList and ageList will be filled with the contents inside of it, then deletes all the contents inside it. The destructor will now delete all the contents of studentName inside the studentList, outputs "Destructor Called", and when the program is done, the j and ageList will be pointing at some random memory.

Table 2-2. Modified Driver Program with Student Lists

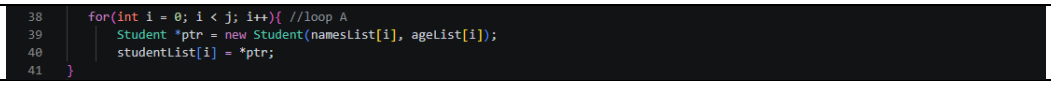
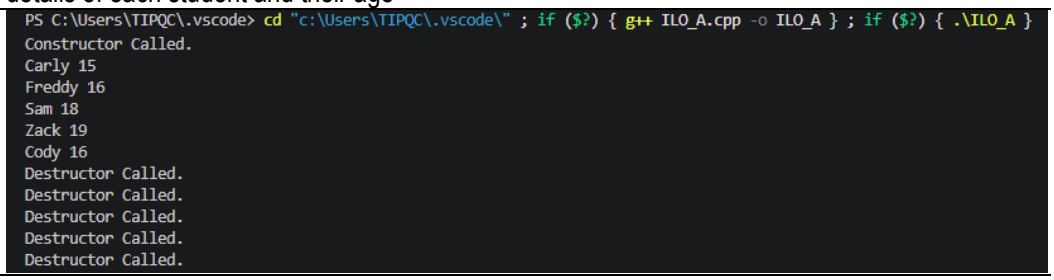
Loop A	
Observation	Loop A accessed the namesList and ageList and puts its details in their corresponding array number. The studentList will now be equal to the Student *ptr.
Loop B	
Observation	Loop B will now print all the details that was given in Loop A and use printDetails to print the details of each student and their age
Output	
Observation	Thie code starts with accessing the class Student and access the content of the studentName and studentAge (John Doe, 18) and outputs "Constructor Called". Then it will go to Loop A and access the contents of namesList and ageList, and puts it in studentList, while Loop will use the printDetails to output the details given in studentList. Then the destrcutor program deletes the contents of the studentList (except studentAge), outputs "Destructor Called" and maes the j and ageList to point at a random memory.

Table 2-3. Final Driver Program

Modifications	
Observation	

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C:

1. Create a class for the fruit and the vegetable classes. Each class must have a constructor, destructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

```cpp
#include <iostream>
// import string modudle to use string data type
#include <string>

// creating class for Fruits
class Fruit {
public:
//declaring variables
    std::string name;
    double price;
    int quantity;

// constrcutor code (with new variable names)
    Fruit(std::string Fname, double Fprice, int Fquantity) {
        name = Fname;
        price = Fprice;
        quantity = Fquantity;
    }

// destructor code
    ~Fruit() {
    }

// copy constructor code
    Fruit(const Fruit &copyFruit) {
        name = copyFruit.name;
        price = copyFruit.price;
        quantity = copyFruit.quantity;
    }

// copy assignment operator
    Fruit& operator=(const Fruit& copy) {
        if (this != &copy) {
            this->name = copy.name;
            this->price = copy.price;
            this->quantity = copy.quantity;
        }
        return *this;
    }

// function to calculate sum using variables price and quantity
    double calculateFruitSum(double price, int quantity) {
        double FruitSum = price*quantity;
        return FruitSum;
    }

};

// creating class for Vegetables
class Vegetable {
public:
//declaring variables
    std::string name;
    double price;
    int quantity;

// constrcutor code (with new variable names)
    Vegetable(std::string Vname, double Vprice, int Vquantity) {
        name = Vname;
        price = Vprice;
        quantity = Vquantity;
    }


// destructor code
    ~Vegetable() {
    }

// copy constructor code
    Vegetable(const Vegetable &copyVegetable) {
        name = copyVegetable.name;
```

```
        price = copyVegetable.price;
        quantity = copyVegetable.quantity;
    }
// copy assignmnet operator
    Vegetable& operator=(const Vegetable& copy) {
        if (this != &copy) {
            this->name = copy.name;
            this->price = copy.price;
            this->quantity = copy.quantity;
        }
        return *this;
    }

// function to calculate sum using variables price and quantity
    double calculateVegetableSum(double price, int quantity) {
        double VegetableSum = price*quantity;
        return VegetableSum;
    }
};
```

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

```
int main() {
// create a GroceryList containing the items from Jenna's Grocery List.
    const size_t j = 4;

     std::string GroceryList[j] = {"Apple", "Banana", "Broccoli", "Lettuce"};
// input all the required details of each product
    Fruit Fruit1("Apple", 10.0, 7);
    Fruit Fruit2("Banana", 10.0, 8);
    Vegetable Vegetable1("Broccoli", 60.0, 12);
    Vegetable Vegetable2("Lettuce", 50.0, 10);

// access each isaved instance
    double PriceList[j] = {Fruit1.Fprice, Fruit2.Fprice, Vegetable1.Vprice, Vegetable2.Vprice};
    int QuantityList[j] = {Fruit1.Fquantity, Fruit2.Fquantity, Vegetable2.Vquantity, Vegetable2.Vquantity};

//display all the details of each item
    std::cout <<GroceryList[0] <<", " <<PriceList[0] <<", " <<QuantityList[0] <<std::endl;
    std::cout <<GroceryList[1] <<", " <<PriceList[1] <<", " <<QuantityList[1] <<std::endl;
    std::cout <<GroceryList[2] <<", " <<PriceList[2] <<", " <<QuantityList[2] <<std::endl;
    std::cout <<GroceryList[3] <<", " <<PriceList[3] <<", " <<QuantityList[3] <<std::endl;
}
```

Output:

```
PS C:\Users\ASUS TUF\Documents\Activities_C++\HOA 2.1> cd "c:\Users\ASUS TUF\Documents\Activities_C++\HOA 2.1\" ;
s } ; if ($?) { .\tets }
Apple, 10, 7
Banana, 10, 8
Broccoli, 60, 12
Lettuce, 50, 10
```

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List

```cpp
// main function / driver code
int main() {
//declare the TotalSum function
    double TotalSum(double FruitSum, double VegetableSum);
// create a GroceryList containing the items from Jenna's Grocery List.
    const size_t j = 4;

    std::string GroceryList[j] = {"Apple", "Banana", "Broccoli", "Lettuce"};
// input all the required details of each product
    Fruit Fruit1("Apple", 10.0, 7);
    Fruit Fruit2("Banana", 10.0, 8);
    Vegetable Vegetable1("Broccoli", 60.0, 12);
    Vegetable Vegetable2("Lettuce", 50.0, 10);

// access each saved instance
    double PriceList[j] = {Fruit1.Fprice, Fruit2.Fprice, Vegetable1.Vprice, Vegetable2.Vprice};
    int QuantityList[j] = {Fruit1.Fquantity, Fruit2.Fquantity, Vegetable1.Vquantity, Vegetable2.Vquantity};

//display all the details of each item
    std::cout <<GroceryList[0] <<", " <<PriceList[0] <<", " <<QuantityList[0] <<std::endl;
    std::cout <<GroceryList[1] <<", " <<PriceList[1] <<", " <<QuantityList[1] <<std::endl;
    std::cout <<GroceryList[2] <<", " <<PriceList[2] <<", " <<QuantityList[2] <<std::endl;
    std::cout <<GroceryList[3] <<", " <<PriceList[3] <<", " <<QuantityList[3] <<std::endl;
// use the calculateSum function to calculate the sum of each fruit and vegetable and display the values
    std::cout <<"Total price of Apple: " <<Fruit1.calculateFruitSum() <<std::endl;
    std::cout <<"Total price of Apple: " <<Fruit2.calculateFruitSum() <<std::endl;
    std::cout <<"Total price of Broccoli: " <<Vegetable1.calculateVegetableSum() <<std::endl;
    std::cout <<"Total price of Lettuce: " <<Vegetable2.calculateVegetableSum() <<std::endl;
// calculate each the total of all fruits and vegetables, then use the TotalSum function to get the total price
    double FruitSum = Fruit1.calculateFruitSum() + Fruit2.calculateFruitSum();
    double VegetableSum = Vegetable1.calculateVegetableSum() + Vegetable2.calculateVegetableSum();
    std::cout <<"Total price of all the items: " <<TotalSum(FruitSum, VegetableSum);
}

// function of TotalSum
double TotalSum(double FruitSum, double VegetableSum) {
    double PriceTotal = FruitSum + VegetableSum;
    return PriceTotal;
}
```

Output:

```
Apple, 10, 7
Banana, 10, 8
Broccoli, 60, 12
Lettuce, 50, 10
Total price of Apple: 70
Total price of Apple: 80
Total price of Broccoli: 720
Total price of Lettuce: 500
Total price of all the items: 1370
```

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned

```cpp
// main function / driver code
int main() {
// initialize each function used
    double TotalSum(double FruitSum, double VegetableSum);
// create a GroceryList containing the items from Jenna's Grocery List.
// Allocate GroceryList and list them one by one as a string
    std::string* GroceryList = new std::string[4];
    GroceryList[0] = "Apple";
    GroceryList[1] = "Banana";
    GroceryList[2] = "Broccoli";
    GroceryList[3] = "Lettuce";

// create a new GroceryList without the lettice
    std::string* NewGroceryList = new std::string[3];
    NewGroceryList[0] = "Apple";
    NewGroceryList[1] = "Banana";
    NewGroceryList[2] = "Broccoli";
```

```
108
109 // input all the required details of each product
110     Fruit Fruit1("Apple", 10.0, 7);
111     Fruit Fruit2("Banana", 10.0, 8);
112     Vegetable Vegetable1("Broccoli", 60.0, 12);
113     Vegetable Vegetable2("Lettuce", 50.0, 10);
114
115 // access each saved instance
116     double PriceList[4] = {Fruit1.Fprice, Fruit2.Fprice, Vegetable1.Vprice, Vegetable2.Vprice};
117     int QuantityList[4] = {Fruit1.Fquantity, Fruit2.Fquantity, Vegetable1.Vquantity, Vegetable2.Vquantity};
118
119 //display all the details of each item
120     std::cout <<NewGroceryList[0] <<", " <<PriceList[0] <<", " <<QuantityList[0] <<std::endl;
121     std::cout <<NewGroceryList[1] <<", " <<PriceList[1] <<", " <<QuantityList[1] <<std::endl;
122     std::cout <<NewGroceryList[2] <<", " <<PriceList[2] <<", " <<QuantityList[2] <<std::endl;
123     std::cout <<"Total price of Apple: " <<Fruit1.calculateFruitSum() <<std::endl;
124     std::cout <<"Total price of Apple: " <<Fruit2.calculateFruitSum() <<std::endl;
125     std::cout <<"Total price of Broccoli: " <<Vegetable1.calculateVegetableSum() <<std::endl;
126     double FruitSum = Fruit1.calculateFruitSum() + Fruit2.calculateFruitSum();
127     double VegetableSum = Vegetable1.calculateVegetableSum();
128     std::cout <<"Total price of all the items: " <<TotalSum(FruitSum, VegetableSum);
129
130
131 //delete the old GroceryList array, and points it to the New Grocery List array
132     delete[] GroceryList;
133     GroceryList = NewGroceryList;
134 }
135
136 // main function of TotalSum
137 double TotalSum(double FruitSum, double VegetableSum) {
138     double PriceTotal = FruitSum + VegetableSum;
139     return PriceTotal;
140 }
```

Output:

```
Apple, 10, 7
Banana, 10, 8
Broccoli, 60, 12
Total price of Apple: 70
Total price of Apple: 80
Total price of Broccoli: 720
Total price of all the items: 870
```

Conclusion:

In conclusion, I properly used arrays, pointers, and dynamic memory allocation. In pointers, they used to point at the address where the variable is stored. You can use & and *, but they have different uses. Arrays are lists that you can edit or take things out of by doing arrayname[n]. Dynamic memory allocation is the process of creating a new memory, using the memory, and deleting it. The steps are to allocate, use, delete, and point it to null using nullptr or NULL.

In the procedure, based on my observations, the first program puts all the details inside studentName and studentAge, and uses the default name and age inside the constructor if no name is given. The second program puts all the information of the array inside the array name, but does not display its contents. The third program uses a for loop to call the printDetails() function so it can print the details of the array for the user to see.

In the supplementary activity, I used a mix of all the topics that were learned in this activity. The code uses constructors, deconstructors, copy constructors, and copy assignment operators inside a class, a function to calculate the sum of fruits, vegetables, and the overall price, arrays that are both initialized one by one and in one line, and uses new and delete to remove a specific item.

This activity is the hardest activity I've ever done, and I relied on AI too much for the program, so I need to improve on every single topic that was involved in this activity, especially dynamic memory allocation. Everything from the procedures to the supplementary activity was too confusing to do and would need a deep understanding, because my knowledge of C++ is not extensive.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance, Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I asked ChatGPT on the overview of the code, so I know the function of each portion of the code, and to help me figure out what code I can add in my program without showing me the actual code. It also helped me clarify the following problem to better understand the situation.**
I used Grammarly to fix my grammar at the conclusion and polish it to make it easy to understand for the most part.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.